

Man-in-the-Middle Attacks via ARP Spoofing on Public Wi-Fi Networks

Murali Krishnan Thirumalai, NETID: mt1171, RUID: 202002076

Department of Electrical and Computer Engineering

Rutgers University - New Brunswick

Email: mt1171@scarletmail.rutgers.edu

Abstract—This paper investigates the execution and implications of Man-in-the-Middle (MITM) attacks through Address Resolution Protocol (ARP) spoofing on public Wi-Fi networks. ARP spoofing enables a third party to intercept communication between two hosts by injecting false ARP responses, thereby redirecting network traffic through the attacker. This work involves practical implementation using Kali Linux, Ettercap, and Wireshark. Experiments were conducted on Windows 11 and an Android emulator connected to a shared local network. Analysis of packet captures and decrypted traffic illustrates the impact of encryption and VPNs on traffic visibility and data security. Key differences in behavior between HTTP and HTTPS traffic were observed, including the limitations of SSLStrip techniques on HSTS-enabled sites. VPN tunneling effectively prevented readable interception, rendering ARP spoofing ineffective in some configurations. This paper explores prevention mechanisms including HTTPS, HSTS, VPNs, and static ARP mappings. Results demonstrate that public networks remain highly vulnerable, and highlight the need for layered security practices in open environments.

I. INTRODUCTION

In today's world of widespread connectivity, public wireless networks have become a cornerstone of digital accessibility. Locations such as cafes, airports, libraries, and university campuses routinely offer open Wi-Fi to visitors, students, and employees. While it is convenient, these public access points frequently lack sufficient encryption or authentication protocols, rendering them attractive targets for cybercriminals. Among the array of techniques utilized to compromise such networks, Man-in-the-Middle (MITM) attacks stand out due to their effectiveness and relative ease of deployment in unprotected environments.

A MITM attack allows an attacker to intercept, monitor, and manipulate communication between two parties which the user and a server without their consent or awareness. One particularly widespread and dangerous variant of MITM is ARP spoofing, which exploits fundamental vulnerabilities in the Address Resolution Protocol (ARP). ARP is responsible for resolving IP addresses to physical MAC addresses within a local area network (LAN), but it lacks any form of authentication or integrity verification. Consequently, any device within the broadcast domain can forge ARP replies, misleading other hosts into associating the attacker's MAC address with a legitimate IP address. This capability enables the attacker to intercept all data intended for the victim or even inject malicious payloads into the traffic flow.

ARP spoofing becomes a much bigger problem on public Wi-Fi. This is because these networks are often set up poorly or are completely open, meaning they do not keep each user's connection separate or encrypt the basic network traffic. The absence of such safeguards permits an attacker to deploy ARP spoofing with minimal resistance, posing serious threats to user privacy, data integrity, and network availability. Once the attacker achieves MITM status, they can engage in a variety of secondary attacks, including but not limited to credential theft, session hijacking, SSL stripping, DNS poisoning, and malware injection.

While various defense mechanisms have been proposed to counteract ARP spoofing including static ARP tables, ARP inspection, and cryptographic ARP extensions, none of them have achieved widespread adoption due to their operational complexity, lack of scalability, or incompatibility with existing protocols. Some methods require the deployment of centralized security appliances, which introduce single points of failure, while others impose substantial processing overheads that degrade network performance. Moreover, the vast majority of public networks lack the infrastructure or administrative control necessary to implement such mitigations effectively.

This project undertakes a systematic investigation into ARP spoofing-based MITM attacks within unsecured public Wi-Fi environments. The goal is to simulate realistic attack scenarios, assess the feasibility and impact of such intrusions, and evaluate the effectiveness of modern countermeasures. The study employs Kali Linux as the attacker platform, leveraging tools such as Ettercap to execute live MITM attacks. Target systems include a Windows 11 virtual machine and an Android emulator, both connected to an open Wi-Fi network to replicate realistic user behavior. Key attack vectors such as SSL stripping, HTTP traffic manipulation, and HTTPS downgrade attempts are explored, with detailed analysis of resulting vulnerabilities and user exposure.

This work shows the critical need for strong and easy-to-use security tools for public Wi-Fi networks. By dissecting the mechanisms of ARP spoofing and demonstrating its real world implications through hands on experimentation, this paper contributes to the growing body of knowledge on wireless network threats. In doing so, it underscores the urgent necessity of improving endpoint and network-level protections to safeguard digital communications in open environments.

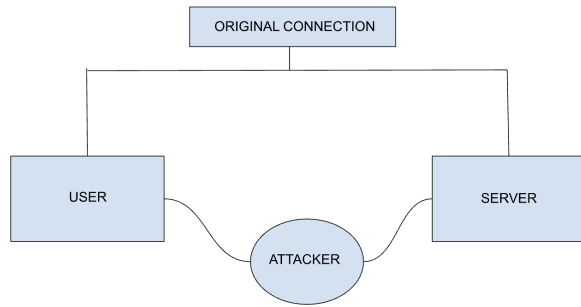


Fig. 1: Mitigation concept: Image Representation of Mitm attack

II. BACKGROUND

A. Address Resolution Protocol (ARP)

The Address Resolution Protocol (ARP) plays a critical role in the operation of IPv4-based local area networks. It serves the fundamental function of resolving 32-bit IP addresses to their corresponding 48-bit MAC (Media Access Control) addresses, which are necessary for actual data transmission over the Ethernet. When a host attempts to communicate with another host on the same subnet, it issues a broadcast ARP request to discover the MAC address associated with a given IP address. All devices on the network receive the ARP request, but only the device owning the queried IP responds with an ARP reply containing its MAC address. The requesting host then caches this mapping in its ARP table to expedite future communication.

However, ARP is inherently insecure due to its lack of authentication mechanisms. The protocol assumes that all hosts on a local network are trustworthy and that responses to ARP queries are legitimate. There is no cryptographic verification, sequence checking, or binding validation in the standard ARP specification (RFC 826). This stateless nature permits any device on the network to send unsolicited ARP replies, regardless of whether an ARP request was issued. These unsolicited packets are often referred to as "gratuitous ARP" and can be used either legitimately or maliciously.

The vulnerability arises because ARP replies are accepted and cached without any validation. An attacker can exploit this behavior by sending forged ARP responses that associate their MAC address with an IP address they do not own—often that of a legitimate gateway or another host. When accepted by the target, the victim updates its ARP cache with incorrect mappings, thereby directing traffic intended for the legitimate destination to the attacker instead. This form of attack is commonly known as ARP spoofing or ARP cache poisoning. Once poisoned, the ARP table of the target device misroutes packets, enabling various downstream attacks, including denial of service, data interception, and traffic manipulation.

Mitigating ARP-based threats is particularly difficult in dynamic environments such as public Wi-Fi networks. Most consumer devices and operating systems do not implement

ARP validation by default, and the dynamic nature of device connections in these networks complicates static configuration. Network-level protections such as dynamic ARP Inspection (DAI) and DHCP snooping exist but are rarely deployed outside of enterprise infrastructures due to hardware and administrative requirements. As a result, ARP remains one of the most exploited protocols for internal network attacks, particularly in unencrypted local environments.

B. MITM via ARP Spoofing

A Man-in-the-Middle (MITM) attack via ARP spoofing leverages the ARP vulnerability to silently intercept and manipulate data between two parties. The attacker inserts themselves into the communication path by sending forged ARP replies to both the victim and the network gateway. The victim is tricked into associating the attacker's MAC address with the IP of the gateway, and simultaneously, the gateway is convinced that the attacker is the victim. Consequently, all traffic flowing between these two nodes is transparently routed through the attacker's device.

This configuration establishes a bidirectional data proxy without alerting either the victim or the gateway. From the perspective of both parties, communication appears uninterrupted, while the attacker silently inspects, logs, modifies, or forwards the data. This stealth is one of the defining characteristics of ARP-based MITM attacks. Unlike more disruptive network attacks, ARP spoofing can persist for extended periods without triggering user suspicion, particularly on open or poorly segmented networks.

The execution of ARP spoofing-based MITM attacks typically involves the use of specialized tools such as Ettercap, Wireshark, etc. These tools automate the sending of continuous ARP replies at high frequency to maintain the poisoned state of the ARP tables, counteracting any legitimate ARP responses that might attempt to restore correct mappings. The attacker can then implement additional features such as packet sniffing, DNS spoofing, or SSL stripping, depending on their goals.

In public Wi-Fi environments, ARP spoofing is facilitated by the lack of client isolation and encryption at the data-link layer. Devices on the same wireless network can freely exchange broadcast and unicast packets, including ARP traffic. Routers that do not implement Wireless Isolation Mode or VLAN segmentation allow any connected client to interact directly with any other device on the network. This topology, combined with open access and transient user sessions, makes public networks a prime target for MITM attacks via ARP spoofing.

Importantly, the success of such an attack does not depend on user interaction or privilege escalation. The attacker does not require administrative access to the victim's device, nor does it require any software or service to be compromised. The manipulation is purely at the network level and exploits default behaviors in network stacks. As such, even security-conscious users are vulnerable if the network environment lacks proper safeguards.

Although HTTPS is widely used to protect web communication through Transport Layer Security (TLS), many websites still expose initial interactions via HTTP before redirecting users to a secure version of the site. This design flaw creates an opportunity for attackers to downgrade secure connections during MITM attacks. One of the most effective techniques for exploiting this weakness is SSL stripping, a method that silently downgrades HTTPS sessions to unencrypted HTTP equivalents.

When a user initiates a connection to a website by typing a domain name into a browser, the initial request is often sent over HTTP by default unless the user explicitly types "https://". Even if the server responds with a redirect to the HTTPS version, this intermediate HTTP request remains vulnerable to interception. SSLStrip exploits this gap by intercepting the initial HTTP request and rewriting the server's HTTPS redirect responses into HTTP. The attacker then establishes a legitimate HTTPS connection with the actual web server on behalf of the user, while serving a plain HTTP version to the victim.

The result is a transparent downgrade of the user session without their knowledge. From the user's perspective, the website behaves normally, but without a secure connection (that is, the browser may not show the padlock icon or show 'Not Secure' in the address bar). Meanwhile, the attacker maintains access to the full content of the session, including any submitted credentials, session cookies, or other sensitive data. Because the connection to the attacker is not encrypted, all user input can be read in plain text.

SSLStrip becomes particularly potent in environments where users are less vigilant about browser indicators, such as in mobile apps or custom browser views where the URL bar is hidden. On top of this, if a website does not implement HTTP Strict Transport Security (HSTS), the browser will accept the HTTP version without complaint, increasing the effectiveness of the attack. HSTS is a browser-side policy that enforces the use of HTTPS for a given domain, but its adoption is not universal and can be bypassed under certain conditions.

From an attacker's standpoint, SSLStrip is relatively simple to implement using tools like Ettercap, which can perform ARP spoofing and SSL stripping simultaneously. Once positioned in the data path via ARP poisoning, the attacker intercepts and modifies all HTTP responses on the fly, effectively acting as a real-time content proxy. Combined with packet capture tools like Wireshark, this setup allows detailed analysis and recording of victim activity across compromised sessions.

SSLStrip illustrates the dangers of incomplete HTTPS implementations and highlights the broader risks of relying solely on transport-layer encryption. It also demonstrates how even well-configured secure websites can be undermined by flaws in user behavior or intermediary network paths. In the context of ARP spoofing-based MITM attacks on public Wi-Fi, SSLStrip provides a powerful mechanism to bypass encryption and compromise sensitive information.

Address Resolution Protocol (ARP) spoofing and associated man-in-the-middle (MITM) threats have long been recognized as critical vulnerabilities in local area networks, particularly within open and untrusted wireless environments. Research in this domain spans several decades and includes both academic studies and penetration testing methodologies used by offensive security professionals.

One foundational study explores the role of ARP spoofing in enabling traffic interception and redirection across insecure networks. They conclude that the absence of authentication in ARP makes it inherently susceptible to deception and hijacking [?]. Many public Wi-Fi deployments offer no client isolation or broadcast domain segmentation, enabling ARP broadcasts and poisoning to propagate freely across connected devices.

Work by Johnson et al. highlights the importance of HTTPS in protecting against credential leakage during such attacks. However, their analysis also notes that improper server configurations, particularly those allowing fallback to HTTP, reintroduce risk even when TLS is technically supported [1]. This includes websites that redirect from HTTP to HTTPS but do not utilize HSTS headers, leaving the initial handshake exposed to downgrade attacks.

In response to these threats, several countermeasures have been proposed and implemented with varying levels of success:

- **Virtual Private Networks (VPNs):** VPNs encapsulate traffic in encrypted tunnels, obscuring payload data from local attackers even if the attacker controls the routing path. Though VPNs are effective, many public users do not employ them consistently.
- **Static ARP Tables:** By manually binding IP addresses to MAC addresses on endpoint devices, static ARP entries can prevent spoofing. However, they are difficult to maintain and scale, especially in dynamic networks with frequently changing devices.
- **HTTP Strict Transport Security (HSTS):** This browser-side policy instructs clients to only interact with designated domains over HTTPS. When deployed correctly, HSTS prevents SSL stripping. Adoption remains inconsistent across websites, especially on legacy systems or test servers.
- **Intrusion Detection and Packet Filtering:** Anomaly-based intrusion detection systems (IDS) can flag rapid or suspicious ARP replies and broadcast behavior as indicative of poisoning attacks. These systems are often limited to enterprise-grade deployments and are not present in most consumer-grade routers or access points.

Other scholarly works propose advanced defenses using behavioral analytics, device profiling, or machine learning to detect spoofing attempts [5]. These mechanisms monitor deviations in MAC-IP bindings or monitor frequency of gratuitous ARP responses. While promising, they often require

infrastructure-level changes, additional overhead, or administrative privileges not available in public network scenarios.

Despite the existence of these mechanisms, ARP spoofing remains highly viable due to its simplicity, effectiveness, and low barrier to execution. It is routinely used in penetration testing exercises to demonstrate network weakness and remains a common tactic among malicious actors in public Wi-Fi environments.

IV. METHODOLOGY

This study replicates and analyzes ARP spoofing-based MITM attacks in two distinct environments using both wired and wireless virtual infrastructure. The design of the experiment aimed to test ARP poisoning in common public network scenarios using commercially available hardware and widely used software tools. Observations were made across both desktop and mobile environments to compare vulnerabilities and response behaviors.

A. Environment Setup

Two primary configurations were employed to simulate and evaluate the MITM attack:

- 1) **Kali Linux Attacker - Targeting Windows 11:** In the first setup, a UTM-based virtual environment was deployed where both the attacker and the victim operated within the same local network range (subnet 192.168.64.0/24). The attacker system ran Kali Linux with root privileges, and the victim system ran Windows 11. Both machines were configured to communicate over a shared bridged adapter, simulating a public Wi-Fi connection without encryption or client isolation.
- 2) **macOS M3 - Kali Linux Targeting Android Emulator:** In the second configuration, a macOS M3 host system ran Kali Linux in a virtual environment. The target was an Android emulator deployed using Android Studio. The emulator used the default AndroidWifi network, which mimics mobile device behavior in open Wi-Fi scenarios. This setup allowed testing of mobile-specific vulnerabilities and interaction with HTTP-based login forms typically found in mobile apps or web views.

Both environments maintained consistent routing configurations, shared DNS, and gateway visibility. Tools such as `arp`, `ipconfig`, and `ifconfig` were used to validate ARP table changes and identify the flow of traffic during poisoning events.

B. Tools Used

Several open-source and penetration testing tools were used to execute, monitor, and analyze the attack:

- **Ettercap:** Ettercap was used as the primary tool for executing ARP poisoning attacks in both graphical and command-line interfaces. The tool allows the attacker to send forged ARP replies to targets on the network, thereby poisoning their ARP caches. In the experiment, Ettercap was configured to target both the default gateway

```
Ethernet adapter Ethernet:
Connection-specific DNS Suffix . : 
IPv6 Address. . . . . : fd0f:54a9:bc24:cccf:16:c0d2:45c:528b
Temporary IPv6 Address. . . . . : fd0f:54a9:bc24:cccf:4d6:6ec:925:f148
Link-local IPv6 Address . . . . . : fe80::990b:8e27:830e:4f04%6
IPv4 Address. . . . . : 192.168.64.6
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 192.168.64.1
PS C:\Users\Muralii Krishnan> arp -a

Interface: 192.168.64.6 --- 0x6
Internet Address      Physical Address      Type
192.168.64.1          82-a9-97-34-af-64     dynamic
192.168.64.7          12-b0-ca-7f-33-9f     dynamic
192.168.64.255        ff-ff-ff-ff-ff-ff     static
224.0.0.22            01-00-5e-00-00-16     static
224.0.0.251           01-00-5e-00-00-fb     static
224.0.0.252           01-00-5e-00-00-fc     static
239.255.255.250       01-00-5e-7f-ff-fa     static
255.255.255.255       ff-ff-ff-ff-ff-ff     static
PS C:\Users\Muralii Krishnan> |

(kali@kali) [~]
$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.64.7 netmask 255.255.255.0 broadcast 192.168.64.255
    inet6 fe80::10b0:caff:fe7f:339f prefixlen 64 scopeid 0<20<link>
    inet6 fd0f:54a9:bc24:cccf:10b0:caff:fe7f:339f prefixlen 64 scopeid 0<0<global>
    inet6 fd0f:54a9:bc24:cccf:4e9:afac:8879:c0fe prefixlen 64 scopeid 0<0<global>
    ether 12:b0:ca:7f:33:9f txqueuelen 1000 (Ethernet)
    RX packets 3 bytes 1010 (1010.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 25 bytes 4446 (4.3 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0<10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

Fig. 2: Network topology used for ARP spoofing attack

and the victim machine (Windows 11 or Android emulator), causing both devices to route their traffic through the attacker's interface. The unified sniffing mode enabled active interception of traffic between the two endpoints. Ettercap's built-in modules allowed live packet filtering, protocol-specific dissection, and real-time analysis. By monitoring the intercepted sessions, the tool displayed HTTP form data, DNS requests, and occasional plaintext credentials transmitted over unsecured connections. The interface also provided detailed logs for each interaction, making it easier to confirm the success of the spoofing and to monitor session flow without needing a separate analyzer for basic inspection.

- **Wireshark:** Wireshark was employed to capture raw packet data before, during, and after the execution of the attack. It was instrumental in verifying the changes in ARP tables and confirming the MITM position of the attacker. Specific filters, including `http`, `dns`, and `arp`, were used to isolate relevant traffic. The tool allowed detailed inspection of Ethernet, IP, and TCP headers, as well as the application layer contents of captured packets. During SSL stripping experiments, Wireshark was used to examine whether traffic was downgraded to plaintext HTTP and whether sensitive information—such as usernames, passwords, or form inputs—could be viewed.
- **Burp Suite:** Burp Suite was used in proxy mode to

inspect and analyze web traffic from the victim system routed through the attacker. The suite captured HTTP requests and responses, providing visibility into form data, cookies, headers, and session variables. When connected via proxy, the browser on the victim machine passed its traffic through Burp, where it could be intercepted and viewed in a structured interface.

The tool also allowed manual tampering with outbound requests and replay of previously captured requests, which is useful for session hijacking demonstrations or parameter manipulation. In the context of MITM experiments, Burp Suite confirmed the ability to view data in transit on HTTP pages, and it also displayed whether HTTPS downgrade attempts via SSLStrip had succeeded. Its intercept and repeater features proved useful for detailed, repeatable traffic inspection.

- **SSLStrip:** SSLStrip was used to downgrade secure HTTPS connections to insecure HTTP transparently. It worked by modifying redirection links in intercepted HTTP responses to prevent the browser from switching to HTTPS. For example, if a user navigated to `http://example.com` and the server attempted to redirect to `https://example.com`, SSLStrip would rewrite this response, causing the client to remain on the unencrypted HTTP version.

Once the connection was downgraded, SSLStrip allowed the attacker to see the full content of transmitted requests and responses in plaintext, including sensitive credentials if entered by the user. The attacker's machine maintained an HTTPS connection with the real server while presenting the client with an HTTP page, effectively acting as a translation layer. This exploit emphasized the importance of HSTS in modern browsers and demonstrated how its absence could leave users exposed.

- **Terminal Utilities and Diagnostic Commands:** A limited set of terminal commands were employed to monitor the attack state and verify whether the spoofing was successful. The command `ipconfig` was used on the Windows machine to identify its IP address and network configuration, while `ifconfig` served a similar purpose on the Kali Linux attacker. The `arp -a` command was essential for inspecting the ARP cache and confirming whether the victim had incorrectly mapped the gateway IP to the attacker's MAC address. Enabling IP forwarding with `echo 1 > /proc/sys/net/ipv4/ip_forward` ensured that the attacker could relay packets between the victim and the gateway without disrupting connectivity. These commands helped validate both the redirection of traffic and the stealth of the attack, providing key insights into how unprotected devices respond to unsolicited ARP replies in a real-world network scenario.

These tools also aided in establishing connectivity and verifying that the attack environment was functioning correctly. Pinging the gateway or target machine was used to ensure consistent communication and to trigger DNS

and ARP traffic for capture. These diagnostic commands were vital in both the initial setup and the validation of ARP poisoning results.

Traffic analysis was conducted during both idle and active browsing states. Login attempts, page visits, and DNS lookups were simulated during attacks to generate real-world packet samples. Browser behaviors were cross-checked for padlock icons, redirection messages, and address bar inconsistencies during the attack timeline.

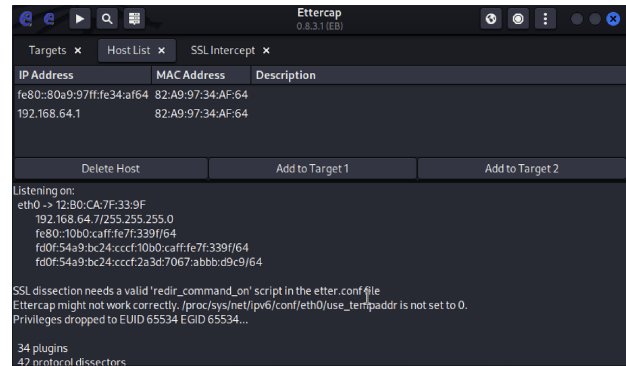


Fig. 3: Ettercap ARP spoofing interface during live attack

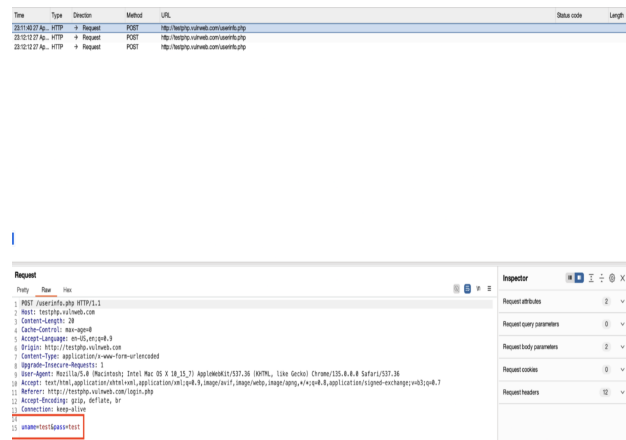


Fig. 4: BurpSuite ARP spoofing interface during live attack capturing Password and Username



Fig. 5: BurpSuite ARP spoofing interface during live attack capturing User credentials

V. RESULTS

A. Windows 11 Target

The ARP spoofing attack was launched successfully from Kali Linux targeting a Windows 11 virtual machine. The vic-

tim device had the IP address 192.168.64.6 and initially associated the gateway 192.168.64.1 with the correct MAC address 82-a9-97-34-af-64. After initiating the attack from Kali Linux (192.168.64.7), ARP poisoning altered the victim's ARP cache, mapping the gateway IP to the attacker's MAC address 12-b0-ca-7f-33-9f. The command `arp -a` executed on the Windows system confirmed this remapping, indicating that traffic intended for the default gateway was now being routed through the attacker's machine.

Network traffic was then captured using Wireshark on Kali Linux. HTTP traffic was fully visible, including credentials submitted via web forms on test sites such as `http://testphp.vulnweb.com`. For instance, POST parameters like `uname=test&pass=test` were clearly extracted from intercepted packets. On the other hand, HTTPS traffic from domains enforcing HSTS remained encrypted and unreadable, as expected. Attempts to strip encryption using SSLStrip were ineffective against modern browsers with proper HTTPS enforcement.

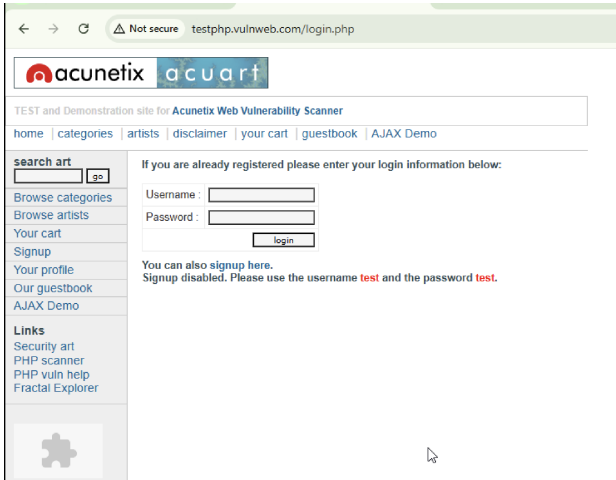


Fig. 6: HTTP test website

To allow the machine to relay traffic between the gateway and the victim, IP forwarding was manually enabled on Kali Linux using the command:

```
echo 1 > /proc/sys/net/ipv4/ip_forward
```

This made sure that communication between the endpoints continued without interruption while still passing through the attacker. As a result, the victim experienced no loss in connectivity, making the attack stealthy and persistent.

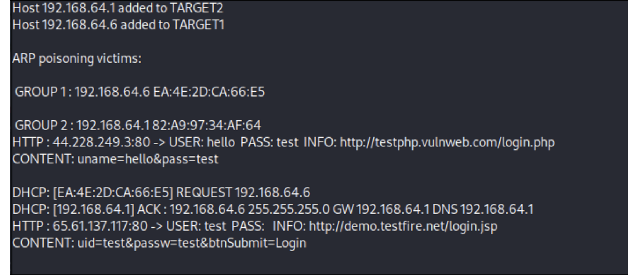


Fig. 7: Captured HTTP credentials from Windows test site

B. ARP Table Observation

Listing 1: Poisoned ARP Table on Victim

Interface : 192.168.64.6	
Internet Address	
Physical Address	Type
192.168.64.1	
12-b0-ca-7f-33-9f	dynamic
192.168.64.7	
12-b0-ca-7f-33-9f	dynamic
192.168.64.255	
ff-ff-ff-ff-ff-ff	static
224.0.0.22	
01-00-5e-00-00-16	static
224.0.0.251	
01-00-5e-00-00-fb	static
224.0.0.252	
01-00-5e-00-00-fc	static
239.255.255.250	
01-00-5e-7f-ff-fa	static
255.255.255.255	
ff-ff-ff-ff-ff-ff	static

As shown above, both the gateway and attacker IPs were pointing to the attacker's MAC address. This confirmed the attacker was effectively impersonating both endpoints simultaneously.

VI. DISCUSSION

The ARP spoofing experiments revealed that man-in-the-middle attacks continue to be feasible and effective in environments lacking robust network protections. During testing, SSLStrip provided partial success when applied to websites that did not enforce HSTS (HTTP Strict Transport Security). On these sites, HTTPS requests were successfully downgraded to HTTP, allowing sensitive data, including usernames and passwords, to be viewed in plaintext. However, on modern web applications that implement HSTS policies or use certificate pinning mechanisms, SSLStrip was unable to perform downgrade attacks. These protections prevented the replacement of HTTPS links with insecure HTTP links and blocked unauthorized certificates, preserving end-to-end encryption.

The experiment also highlighted the limitations of attacks when facing contemporary security configurations. Applications and browsers that adhere to HTTPS-only policies resisted

manipulation. Even where initial redirects from HTTP to HTTPS occurred, HSTS policies stored in browsers ensured secure connections. Similarly, services employing TLS certificate pinning prevented forged certificates from being accepted, cutting off the attacker’s ability to perform content injection or view encrypted traffic.

Virtual Private Networks (VPNs) emerged as the most effective mitigation. When VPNs were enabled on the victim machine, all traffic was encrypted, including that originating from non-secure HTTP sites. Wireshark packet captures confirmed that no credentials or sensitive information could be seen, as payloads were encapsulated and protected by the VPN tunnel. The attacker was only able to view metadata such as DNS requests or destination IP addresses, none of which contained private information. This underscores the importance of transport-level encryption regardless of the visited site’s configuration.

These findings support the critical need for defense-in-depth strategies. Relying solely on HTTPS or endpoint firewalls is insufficient. A layered security approach involving VPNs, client-based HSTS enforcement, and ARP monitoring can significantly reduce exposure. Furthermore, users connected to public networks should be aware of the heightened risk of packet manipulation and impersonation attacks.

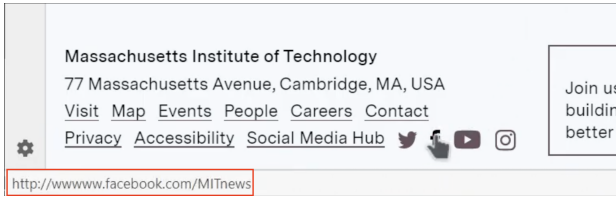


Fig. 8: SSLStrip interception of encrypted credentials (HSTS site)

A. VPN-Protected Traffic Analysis

Packet captures conducted during the use of a VPN on the victim system revealed that nearly all traffic was encrypted, leveraging protocols such as QUIC and TLSv1.3. In Wireshark, each captured packet was labeled as a ”Protected Payload” with no decipherable application-level content. The attacker was unable to view HTTP requests, form submissions, or session cookies—only metadata such as destination IPs and packet sizes were exposed.

Representative encrypted packets observed in Wireshark include:

- 23.200.0.27 → 192.168.64.3 QUIC
1292 Protected Payload
- 192.168.64.3 → 23.43.85.20 QUIC 78
Protected Payload
- 192.168.64.3 → 20.189.173.8 TLSv1.3
Application Data

These entries confirm that VPNs encapsulate all transmitted data in secure tunnels, effectively neutralizing ARP-based interception attempts. The encrypted nature of payloads persisted throughout both idle and active sessions, including

during visits to test websites and interaction with browser-based services.



Fig. 9: Wireshark view of VPN-protected traffic

These entries demonstrate that VPNs wrap all transmitted data in encrypted tunnels, defeating ARP-based interception attempts. The encrypted payloads were consistent even across idle and active browsing periods, including visits to test websites and interaction with common services.

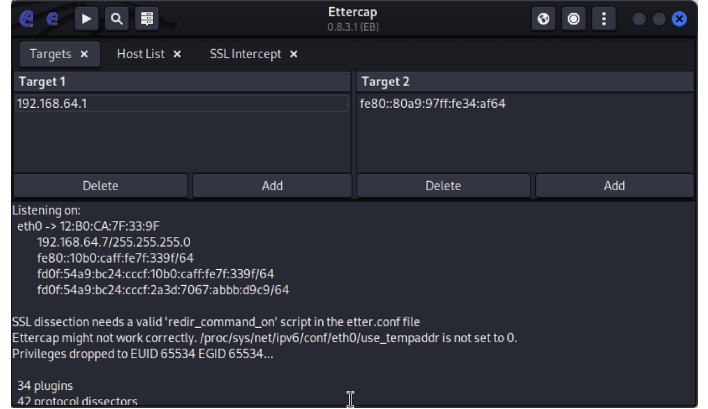


Fig. 10: Ettercap view of VPN-protected traffic showing encrypted payload

VII. MITIGATION STRATEGIES

ARP spoofing is a serious vulnerability, particularly in environments where open or misconfigured networks are present. Several effective mitigation strategies can be adopted to minimize the risk:

- **Utilize VPNs:** VPN services encrypt all network traffic between the client and VPN server. This makes the data unreadable to attackers who may have infiltrated the same network segment.
- **Enable Static ARP Tables:** Defining fixed IP-to-MAC address mappings prevents endpoints from accepting unsolicited ARP replies. While harder to manage on large networks, this is effective in critical infrastructure setups.
- **Deploy Dynamic ARP Inspection (DAI):** Switches and routers configured with DAI validate ARP messages against known mappings before forwarding them. This prevents poisoning attempts from reaching clients.
- **Monitor with IDS/IPS:** Intrusion detection systems (IDS) and intrusion prevention systems (IPS) can detect abnormal ARP traffic patterns and raise alerts or block spoofed packets.
- **Avoid Public Wi-Fi:** If feasible, users should refrain from using public Wi-Fi networks. Personal mobile hotspots are generally more secure.
- **Favor Secure Websites:** Prioritize access to domains with enforced HTTPS, valid TLS certificates, and HSTS

headers. Browsers should not accept HTTP versions of these sites.

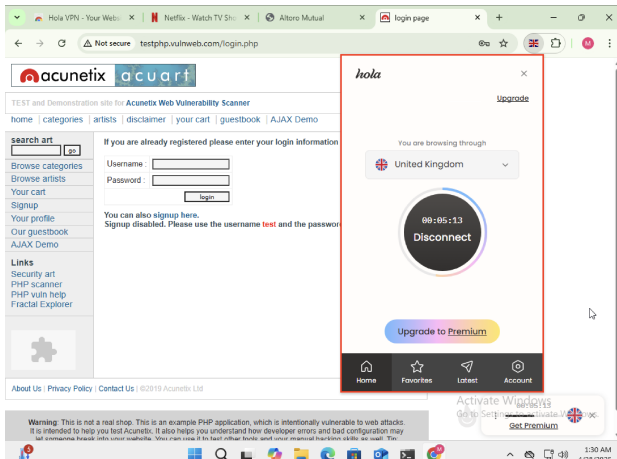


Fig. 11: Using HTTP Test Website with VPN turned ON

VIII. CONCLUSION

The execution of ARP spoofing attacks on public and virtual networks reinforces the importance of baseline network security hygiene. This paper demonstrated how malicious actors can position themselves as intermediaries by poisoning ARP tables, effectively rerouting traffic through their devices without the knowledge of users. Tools such as Ettercap were used to execute the spoofing, while Wireshark provided visibility into intercepted traffic. In cases where victims accessed unencrypted HTTP services, attackers were able to extract sensitive information such as usernames and passwords. This validated the effectiveness of classic MITM attacks in modern public Wi-Fi environments where protections are minimal or absent.

SSLStrip techniques were also tested to downgrade HTTPS traffic, with varying success. Although this method can expose data in weakly secured sites, widespread adoption of HTTPS, HSTS, and certificate pinning has limited its effectiveness. Modern browsers actively enforce secure connections and block unauthorized certificate substitutions, preventing attackers from intercepting encrypted content even after ARP spoofing succeeds. This shift underscores a growing resistance to traditional MITM methods at the application layer.

Empirical results confirm that devices operating on unsecured networks without encryption or VPNs remain vulnerable to credential theft, session hijacking, and traffic inspection. However, enabling VPNs or using websites that implement strict HTTPS configurations can mitigate these risks. Packet captures from VPN-protected sessions showed complete encryption of application data, limiting the attacker's visibility to metadata such as IP headers and packet lengths. The use of layered defenses—including static ARP mappings, dynamic ARP inspection, and intrusion detection systems—further reduces attack feasibility.

In conclusion, while ARP spoofing remains a viable attack vector, especially in public or poorly monitored networks, its

success depends heavily on the absence of modern protective mechanisms. User behavior, network design, and endpoint configuration all contribute to security outcomes. Ongoing education and proactive configuration are essential for maintaining the confidentiality and integrity of communications, particularly in environments where MITM attacks are plausible.

REFERENCES

- [1] L. Johnson, "HTTPS Everywhere and the Limits of SSLStrip," *Proc. Int. Conf. on Secure Systems*, 2021.
- [2] Ettercap Project. Ettercap Documentation. Available: <https://www.ettercap-project.org>
- [3] Bettercap Project. Bettercap Wiki. Available: <https://www.bettercap.org>
- [4] S. M. Morsy and D. Nashat, "D-ARP: An Efficient Scheme to Detect and Prevent ARP Spoofing," *IEEE Access*, vol. 10, pp. 49142–49153, 2022, doi: 10.1109/ACCESS.2022.3172329.
- [5] A. Prasad and S. Chandra, "Defending ARP Spoofing-based MitM Attack using Machine Learning and Device Profiling," in *Proc. 2022 Int. Conf. on Computing, Communication, and Intelligent Systems (ICCCIS)*, Greater Noida, India, 2022, pp. 978–982, doi: 10.1109/ICCCIS56430.2022.10037723.
- [6] Y. Zhao, R. Guo and P. Lv, "ARP Spoofing Analysis and Prevention," in *Proc. 2020 5th Int. Conf. on Smart Grid and Electrical Automation (ICSGEA)*, Zhangjiajie, China, 2020, pp. 572–575, doi: 10.1109/ICSGEA51094.2020.00130.
- [7] H. A. S. Adjei, T. Shunhua, G. K. Agordzo, Y. Li, G. Peprah and E. S. A. Gyarteng, "SSL Stripping Technique (DHCP Snooping and ARP Spoofing Inspection)," in *Proc. 2022 24th Int. Conf. on Advanced Communication Technology (ICACT)*, PyeongChang, Korea, 2022, pp. 187–193, doi: 10.23919/ICACT53585.2022.9728961.
- [8] H. A. S. Adjei, T. Shunhua, G. K. Agordzo, Y. Li, G. Peprah and E. S. A. Gyarteng, "SSL Stripping Technique (DHCP Snooping and ARP Spoofing Inspection)," in *Proc. 2022 24th Int. Conf. on Advanced Communication Technology (ICACT)*, PyeongChang, Korea, 2022, pp. 187–193, doi: 10.23919/ICACT53585.2022.9728961.
- [9] B. Pingle, A. Mairaj and A. Y. Javaid, "Real-World Man-in-the-Middle (MITM) Attack Implementation Using Open Source Tools for Instructional Use," in *Proc. 2018 IEEE Int. Conf. on Electro/Information Technology (EIT)*, Rochester, MI, USA, 2018, pp. 0192–0197, doi: 10.1109/EIT.2018.8500082.
- [10] D. R. Rupal, D. Satasiya, H. Kumar and A. Agrawal, "Detection and prevention of ARP poisoning in dynamic IP configuration," in *Proc. 2016 IEEE Int. Conf. on Recent Trends in Electronics, Information & Communication Technology (RTEICT)*, Bangalore, India, 2016, pp. 1240–1244, doi: 10.1109/RTEICT.2016.7808030.
- [11] F. Callegati, W. Cerroni and M. Ramilli, "Man-in-the-Middle Attack to the HTTPS Protocol," *IEEE Security & Privacy*, vol. 7, no. 1, pp. 78–81, Jan.–Feb. 2009, doi: 10.1109/MSP.2009.12.
- [12] F. Fayyaz and H. Rasheed, "Using JPCAP to Prevent Man-in-the-Middle Attacks in a Local Area Network Environment," *IEEE Potentials*, vol. 31, no. 4, pp. 35–37, July–Aug. 2012, doi: 10.1109/M-POT.2012.2187102.
- [13] N. Xhemajli and Z. Tafa, "Mobile Proxy in Public WiFi Networks: A Tool Against MITM Attacks," in *Proc. 2024 13th Mediterranean Conf. on Embedded Computing (MECO)*, Budva, Montenegro, 2024, pp. 1–5, doi: 10.1109/MECO62516.2024.10577803.